

第2章 程序设计基础

- ◆ 掌握Python的基本运算
- ◆ 掌握变量、表达式和语句
- ◆ 掌握Python的基本数据类型
- ◆ 掌握Python的基本输入和输出

- Python语法常识
- Python语句的书写规则
- 变量
- 基本数据类型
- 运算符与表达式
- 常用函数

Python程序的组成

Python的程序由包、模块（即一个Python文件）、函数、类和语句组成。



(1) 命名规则

- **变量名、包名、模块名**通常采用**小写字母**开头，如果名称中包含**多个单词**，一般采用**第一个单词全部小写**，后面**每一个单词首字母大写的驼峰表示法**，如myBook。也可以采用**下划线_分隔的全部小写形式**，如student_name。一些**不变的对象**，建议使用**全大写**方式，如PI。
- **类名**采用**首字母大写**，**多个单词**使用**驼峰表示法**，如BookInfo。
- **函数名**一般采用**小写字母**，**多个单词**使用**驼峰表示法**。

(2) 代码缩进

- 使用**Tab键和空格**来进行代码缩进，但是不要**混用Tab和空格来缩进**。
- **Python**中的缩进代表程序块的**作用域**，如果采用了错误的**代码缩进**，会导致程序**抛出异常**。

(3) 空格

函数或者语句块之间使用空格行来分隔，以分开两段不同功能的代码块，增强可读性。运算符两侧建议使用空格进行分隔。

(4) 注释

注释有助于对程序的理解和团队合作开发，对函数、类一定要添加功能性、使用性注释说明，对于复杂的算法也要适当注释。

(5) 每个import语句只导入一个模块，尽量避免一次导入多个模块。

(6) 如果一行语句太长，可以在行尾使用续行符“\”，在下一行继续写代码。

(7) 适当使用异常处理结构提高程序的容错性和健壮性（鲁棒性）。

必须先了解的几个Python语法常识。

1. 代码注释方法

1) Python的单行注释语句用#开始，从#开始一直到末尾的部分是注释部分，。

2) 如果要进行多行的注释可以使用三个单引号（'''）或者双引号（"""）将注释内容包围。单引号和双引号在使用上没有本质的差别。

In [36]: ▶ *#人生苦短，我用Python*

```
'''
    This is my first python programm
    created on 2021-2-19
    @author lily
'''

'''
    This is my first python programm
    created on 2021-2-19
    @author lily
'''

x=1
print(x)
```

1

2. 缩进语句

- Python最具特色的就是使用缩进来表示代码块，不需要使用大括号{}，但是同一个代码块的语句必须包含相同的缩进空格数（4个空格）。
- 一个模块的界限，完全由每行的首字符在这一行的位置来决定的。
- 代码缩进有两种方式，一种是采用制表符(Tab键)，另一种是采用若干个空格。
- Python的一些特殊语句例如条件语句、循环语句等规定要向右边缩进。

代码缩进，判断用户从键盘输入的数字是正数、负数还是0。

```
In [2]: ▶ #接受键盘上输入的一个数字
x=input("请输入一个数字：")
#判断从键盘上输入的数字是否是正数
x=float(x)
if x>0:
    print('您输入的数字是正数')
elif x==0:
    print("您输入的数字是0")
else:
    print("您输入的数字是负数")
```

```
请输入一个数字：-3
您输入的数字是负数
```

3. 多行语句

Python程序一般一行写一条语句，相同性质的语句左对齐，例如下面三条输出语句：

```
In [15]: ▶ x="Hi!"  
          y="Everyone"  
          z=x+y  
          print(z)
```

Hi!Everyone

3. 多行语句

这种语句也可以写在同一行，语句之间使用分号分开，上面三条语句写成：

```
In [16]: x="Hi!";y="Everyone";z=x+y;print(z)  
Hi!Everyone
```

一般建议把多条短的语句写在一行，大部分语句还是一条语句占一行。

4. 输出语句

Python使用print语句进行输出，它可以输出任意多个数据，数据之间用逗号分开，例如：

```
In [19]: ▶ x=1;y=2  
print(1, 2, 1+2, "x+y=", x+y)  
  
1 2 3 x+y= 3
```

5. 中文编码

在Python中采用Unicode编码，所有的英文字符与汉字都使用2个字节表示，因此处理汉字与处理英文字符是一样的，没有太多区别。

在Python中使用函数`ord(x)`查看x字符的编码，编码占2个字节，因此一般再使用`hex`函数把编码转为十六进制显示。

```
In [21]: ▶ print(hex(ord("A")))
          print(hex(ord("我")))
```

0x41

0x6211

6. 关键字（保留字符）

关键字是Python语言本身使用的标识符，被Python语言保留，每个关键字都有其特殊的意义，因而**不能用作标识符**，否则容易产生编译错误。

表2-1 Python中的关键字

False	True	None	and	or
not	as	with	from	import
while	for	in	continue	break
try	raise	finally	class	def
global	nonlocal	del	yield	except
if	else	elif	pass	return
lambda	async	assert	await	break

表 1-3-1 Python 常用保留字符

保留字符	说明
and	用于表达式运算，逻辑与操作
as	用于类型转换
assert	断言，用于判断变量或条件表达式的值是否为真
break	中断循环语句的执行
class	用于定义类
continue	继续执行下一次循环
def	用于定义函数或方法
del	删除变量或者序列的值
elif	条件语句 与 if else 结合使用
else	条件语句 条件语句，与 if,elif 结合使用。也可以用于异常和循环使用
except	except 包括捕获异常后的操作代码，与 try, finally 结合使用
exec	用于执行 python 语句
for	循环语句
finally	用于异常语句，出现异常后，始终要执行 finally 包含的代码块。 与 try,except 结合使用
from	用于导入模块，与 import 结合使用
global	定义全局变量
if	条件语句，与 else, elif 结合使用
import	用于导入模块，与 from 结合使用

in	判断变量是否存在序列中
is	判断变量是否为某个类的实例
lambda	定义匿名函数
not	用于表达式运算，逻辑非操作
or	用于表达式运算，逻辑或操作
pass	空的类，函数，方法的占位符
print	打印语句
raise	异常抛出操作
return	用于从函数返回计算结果
try	包含可能会出现异常的语句，与 except, finally 结合使用
while	循环语句
with	简化 Python 的语句
yield	用于从函数依次返回值

查看Python中的所有关键字（保留字符）

```
In [22]: ▶ import keyword  
print(keyword.kwlist)
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except',  
'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'whi  
le', 'with', 'yield']
```

检查是否是Python中的关键字（保留字符），用
`keyword.iskeyword (word)`

```
In [25]: ▶ keyword.iskeyword("and")
```

```
Out[25]: True
```

```
In [26]: ▶ keyword.iskeyword("String")
```

```
Out[26]: False
```

7. 标识符

用来表示某个实体的符号，由**字母、下划线和数字**组成，**且不能以数字开头**。

a_int
b52
str1_strname
funcl



99var
It's0K
For
X.Y



Python标识符**区分大小写**。例如，ABC和abc视为不同的名称。

使用标识符应注意：

- (1) 见名知义：通过变量名就知道变量值的含义。
- (2) 开头和结尾都使用下划线的情况应该避免： **`__sum__`**
- (3) 不能和关键字重名。
- (4) 不能以数字开头。
- (5) 标识符不宜太长。
- (6) 不建议使用系统内置的模块名、类型名或函数名，以及已导入的模块名及其成员名作为变量名。

单独的下划线（_）用于表示上一次的运算的结果：

```
In [27]: 2*3
```

```
Out[27]: 6
```

```
In [28]: _*10
```

```
Out[28]: 60
```

标识符可以被用作变量名、函数名、类名、模块名等的命名。

标识符命名规则

变量名、包名、模块名通常采用小写字母开头。

类名首字母采用大写字母。

函数名一般采用小写字母。

多个单词使用驼峰表示法。

常量所有的字母都用大写，单词用下划线分开。

最重要的命名规则是，选取的名称应该能够清楚地说明该变量、函数，类、模块等所包含的意义。

内置函数

内置函数

函数就是程序中一段包装起来的具有特定功能的代码。
内置函数(也称内建函数)指的是不需要导入任何模块即可直接使用的函数。

函数通过函数名和参数列表进行调用, 通过返回值向外部返回结果。

类型转换函数: `chr(97)` `ord('a')` `str(3.15)`
`int(3.3)`

数学运算函数: `abs(x)` `max(3, 2, 5, 1)` `pow(x, y)`

模块与函数

模块与函数

Python使用**模块**将代码封装起来。除了Python的内置函数之外，Python标准库所提供的函数均被封装在各个模块中。

要**调用模块中的函数(非内置函数)**，需要在**代码顶部**使用**import语句**导入该模块。

- (1) `import 模块名`
- (2) `from 模块名 import 函数名`

Python语句的书写规则

1. 语句需要从第一列开始，前面不能有任何空格，否则会产生语法错误。
2. 程序的注释可以出现在任何位置，以符号“#”开始，到行末结束。
3. 复合语句包括：if语句、while语句、for语句、try语句、with语句、函数定义、类定义等。
4. 缩进：每级缩进用4个空格；连续行的折叠元素应该对齐。
5. 导入：通常应该在单独的行中导入(Import)。

```
import os
```

这样也是可以的

```
from subprocess import Popen, PIPE
```

import通常被放置在文件的**顶部**，仅在**模块注释**和**文档字符串**之后，在模块的**全局变量**和**常量**之前。

● Python语句的书写规则

6. 简单语句包括：表达式语句、赋值语句、assert 语句、pass 空语句、del语句、return语句、yield 语句、raise 语句、break 语句、continue 语句、import 语句、global 语句、nonlocal语句等。
7. 若同一行内放置多个语句，可以在语句中间采用分号（;）进行分隔。
8. 如果语句太长，可以使用续行符（\），但三引号定义的字符串（"""..."""）、元组（（...））列表（[...]）、字典（{...}），可以放在多行，无需使用续行符（\）。
9. 程序有时需要一个空语句来占位，而不是一个空行，空语句用pass语句表达。
10. 语句不需要分号（;）作为结束符，一般一行一条语句。
11. 注释：同代码不一致的注释比没注释更差. 当代码修改时, 始终**优先更新注释！**

12. Python中有许多内置对象可供编程者直接使用，如数字、字符串、列表、集合以及内置函数；
13. 非内置对象需要导入模块后才能使用，如math模块中的正弦函数`sin()`，random模块中的随机数产生函数`random()`等。

- 对象是Python语言中最基本的概念，在Python中处理的一切都是对象。
- Python中的对象就是编程中把数据和功能包装后形成的一个对外具有特定交互接口的内存块。
- 对象是对数据的抽象，Python中所有的数据都被表示成对象，或者对象之间的关系，所有的数据以对象的形式存在。一串字符、一个数字或者一个集合都是对象，如：“hello world”、2、42.56、{1, 2, 3}等。

➤ 每个对象都有三个属性分别是：

- **身份** (identity)，就是对象在内存中的地址；
- **类型** (type)，用于表示对象所属的数据类型(类)，对象的类型决定了对象可以存储什么类型的值，有哪些属性和方法，可以进行哪些操作；
- **值** (value)，对象所表示的数据。

变量就是程序为方便地引用内存中的值而设置的一种让使用者容易识别的名称。在Python中，变量是用一个变量名表示，变量名的命名规则：

- 变量名只能是字母、数字或下划线的任意组合。
- 变量名的第一个字符不能是数字。
- Python关键字不能声明为变量名。
- 永远不要用字符 ‘1’ (小写字母el (就是读音, 下同)), ‘0’ (大写字母oh), 或 ‘I’ (大写字母eye) 作为单字符的变量名。在某些字体中, 这些字符不能与数字1和0分开。当想要使用 ‘1’ 时, 用 ‘L’ 代替它。

- Python中的变量是**不需要声明数据类型的**，变量的“类型”是所指的内存中被赋值对象的类型，**变量类型会根据所赋值的类型自动推断**。例如，int类型上可以进行四则运算。
- **变量在使用前，必须赋初值。**
- **变量在第1次赋值时被创建**，再次出现时直接使用。

- 同一变量可以反复赋值，而且可以是不同类型的变量，这也是Python语言称之为动态语言的原因。
- Python允许同时为多个变量赋值。
- **变量引用了对象。** 当在表达式中使用变量时，变量被其引用的对象替代。
- **Python中各种变量存储的不是值，而是值的引用。**

- 变量的**创建**：直接给变量赋值，无需其它操作。

```
In [39]: ▶ brower = 'Google'    #字符串类型
          brower = 100           #整数类型
          brower = 123.45        #浮点数类型
          brower = 2 + 3j         #复数类型
          brower, count, addsum = 'Google', 100, 123.45
          print(brower, count, addsum)
```

Google 100 123.45

```
In [40]: ▶ x=0; y=0; z=0        #多个语句间用分号分隔
          a, b = 1, 2            #采用多变量同时赋值
          c=d=3                  #采用链式赋值方式
          print(x, y, z, a, b, c, d)
```

0 0 0 1 2 3 3

➤ 与变量属性相关的内置函数：

(1) `type()` 函数：一般形式：**`type(变量名)`**，其作用是查询变量的数据类型。

```
In [50]: ▶ num=5  
          type(num)
```

```
Out[50]: int
```

```
In [51]: ▶ str="Hello World!"  
          type(str)
```

```
Out[51]: str
```

```
In [52]: ▶ z=True  
          type(z)
```

```
Out[52]: bool
```

➤ 与变量属性相关的内置函数：

(2) `id()` 函数：一般形式：`id(变量名)`，其作用是获取变量的内存地址。

```
In [53]: ▶ num=1  
          id(num)
```

```
Out[53]: 140713100257056
```

```
In [54]: ▶ str="Hello World!"  
          id(str)
```

```
Out[54]: 1972417216432
```

➤ 与变量属性相关的内置函数：

(3) `isinstance()` 函数：一般形式：**`isinstance`** (**对象**, **类型名**), 其作用是判断一个数值是否是某种数据类型。

```
In [55]: ▶ num=3  
isinstance(num, int)
```

Out[55]: True

```
In [60]: ▶ isinstance(num, bool)
```

Out[60]: False

```
In [61]: ▶ isinstance(num, (bool, int, str, float))
```

Out[61]: True

➤ 变量的删除

- Python具有**自动内存管理**功能，对于没有指向值的变量，Python会自动将其删除。Python会跟踪所有的变量，并自动删除不再有指向值的变量。因此，Python程序员一般情况下不需要太多考虑内存的管理问题。
- 通过显式使用**del**命令，可以删除不需要的变量，或者显式关闭不再需要访问的资源。

● 变量

- 变量的**删除**：通过del语句删除对象的引用，也就是删除之前所创建的对象。

具体语法：

`del var1[, var2[, var3[...., varN]]]`

In [63]:

```
▶ a=2  
del a  
print(a)
```

```
-----  
NameError                                Traceback (most recent call last)  
<ipython-input-63-ab5d864a017e> in <module>  
      1 a=2  
      2 del a  
>>> 3 print(a)  
  
NameError: name 'a' is not defined
```

➤ 常量的表示:

常量是内存中用于保存固定值的单元，在程序中常量的值不能发生改变。Python只能用变量来表示常量，通常的做法是采用大写字母来表示在程序里的常量。

```
In [64]: ► PI =3.141592653  
          print(PI)  
          PI=3  
          print(PI)  
  
          3.141592653  
          3
```

注：PI是人为指定的一种作为常量使用的变量，其本质还是个变量，能够被重新赋值。

- 数据类型不仅决定了对象在内存中的存储方式，而且决定了可以在对象上附加的操作。基于不同的数据类型，程序可以实现复杂的功能。
- Python语言中，所有对象都有一个数据类型，Python数据类型定义为一个值的集合以及定义在这个值集上的一组运算操作。一个对象上可执行且只允许执行其对应数据类型定义的操作。

➤ Python中有六个标准的数据类型：

- number（数字）
- string（字符串）
- list（列表）
- tuple（元组）
- dictionary（字典）
- set（集合）

其中，不可变数据类型有：number、string、tuple；
可变数据类型有：list、dictionary、set。

Python3支持的数字类型有int（整数）、float（浮点数）、bool（布尔型）、complex（复数）四种类型。

- **数字**（包括布尔型）、**字符串**和**空类型**称为**原子（标量）类型**，它每次只能**存储单个对象类型**。
- **列表、元组、字典和集合**是Python中的**内置容器类型**，容器是用来存放基本对象或者其它容器对象的一种类型，它可以容纳多个**对象类型**。**不同容器类型**之间的最主要区别是**单个元素的访问方式以及运算符定义方式的不同**。
- Python提供了对多种数据类型的强大支持，其中**数字、字符串、列表、元组、字典、集合**等称为**标准数据类型**，其它的类型称为**内建类型**。

基本数据类型

对象类型	名称	实例	对象类型	名称	实例
数字	int、float、complex	123, 3.1415, 3+4j	集合	set	{"a","b","c"}
字符串	str	" hello " , '12', '"语言"'	元组	tuple	(1,2,3)
字节串	bytes	b'\xd6\xd0\xb9\xfa'	文件		fn=open("name.txt",'r')
列表	list	[1,2,3], ['a','b',['c','d']]	布尔型	bool	True、 False
字典	dict	{1:'a',2:'b'}	空类型	NoneType	None

- Python支持四种基本**数字类型**，分别是**整型**、**布尔型**、**浮点型**、**复数型**，其中前两种是整型类型。
- Python中的**数字是无符号**的，所有的数字不包括符号“-”，符号“-”被看做一元运算符“-”。
- **浮点数**由于表示方式的限制，在进行运算时**很少返回精确的预期结果**。

- 数字属于Python的不可变对象，修改整型变量值的时候并不是真正修改变量的值，而是修改变量使其指向新值所在的内存地址。
- 为了增强数字的可读性，可以在数字中间位置使用单个下划线作为分隔。
- 在Python中，数字类型变量所表示的范围可以是无穷大，只要内存空间足够。和其它语言一样，Python也可以对数据类型进行等价转换。

➤ Python包括4种内置的数字数据类型。

(1) `int`整型。用于表示整数，如12，1024， 10。

(2) `float`浮点型。用于表示实数，如3.14，1.2，
2.5e2 ($= 2.5 \times 10^2 = 250$) 。

(3) `bool`布尔型。`bool`布尔型对应两个布尔值：
`True`和`False`，分别对应1和0。

(4) `complex`复数型。其表示有两种：一种是`a+bj`，
另一种是`complex(a, b)`，例如`3 + 4j`可表示成
`complex(3, 4)`。

基本数字数据类型-整型数据

- 整型数据即整数，不带小数点，可以有正号或者负号。
- 在Python3.x中，整型数据在计算机内的表示没有长度限制，其值可以任意大。
- 为了增强数字的可读性，可以在数字中间位置使用单个下划线作为分隔。

```
In [76]: ▶ b = 8888_888_888_888_888
          c = b ** 5
          print(c)
```

```
Out[76]: 152415787504953525750053345778750190521
```

● 基本数字数据类型-整型数据

- **十进制整数**：由0到9组成。
- **二进制整数**：以0b或0B为前缀，由0和1组成。 比如0b10，表示二进制的10，即10进制的2。
- **八进制整数**：以0o或0O为前缀，其后由0至7的数字组成。
比如0o10，表示八进制的10，即十进制的8。
- **十六进制整数**：以0x或0X开头，其后由0至9的数字和a至f字母或A至F字母组成。 比如0x10，表示十六进制的10，即十进制的16。

根据前缀来区分各种进制数

➤ Python中常用的内置进制转换函数：

- **int(二进制整数型字符串,2)**，二进制转换成十进制
- **int(八进制整数型字符串,8)**，八进制转换成十进制
- **int(十六进制整数型字符串,16)**，十六进制转换成十进制
- **bin(整数)**，将整数转换为对应的二进制字符串
- **oct(整数)**，将整数转换为对应的八进制字符串

● 基本数字数据类型-整型数据



➤ Python中常用的内置进制转换函数:

```
In [70]: ► int('11', 2)
```

```
Out[70]: 3
```

```
In [72]: ► int('71', 8)
```

```
Out[72]: 57
```

```
In [73]: ► int('0f', 16)
```

```
Out[73]: 15
```

```
In [74]: ► bin(5)
```

```
Out[74]: '0b101'
```

```
In [75]: ► oct(9)
```

```
Out[75]: '0o11'
```

基本数字数据类型-浮点型数据



实数又称浮点数，一般有两种表示形式：

- **十进制小数形式：** 由数字和小数点组成（必须有小数点）。
- **指数形式：** 用科学计数法表示的浮点数，用字母e（或E）表示以10为底的指数，e之前为数字部分，之后为指数部分。

注意： e（或E）前面必须有数字，后面必须是整数。

例如： e-5, 1.2E-3.5, 1e



浮点数由于表示方式的限制，在进行运算时很少返回精确的预期结果。

```
In [36]: ▶ x, y=0.4, 0.3  
          print(x-y)  
  
          0.10000000000000003
```

● 基本数字数据类型--布尔型数据

- Python的布尔类型有两个值：**True**和**False**，分别表示逻辑真和逻辑假。分别对应了数字的1和0，因此布尔型是整数类型的子类型。

```
In [77]: ▶ type(True)
```

```
Out[77]: bool
```

```
In [78]: ▶ True==1
```

```
Out[78]: True
```

```
In [79]: ▶ False==0
```

```
Out[79]: True
```

布尔类型还可以与其它数据类型进行逻辑运算，Python规定：0、空字符串、None为False，其他数值和非空字符串为True。

```
In [80]: ► 0 and True
```

```
Out[80]: 0
```

```
In [83]: ► None or True
```

```
Out[83]: True
```

● 基本数字数据类型—复数

- 复数由实数部分和虚数部分构成，可以用 $a + bj$ 或者`complex(a, b)`表示，复数的实部 a 和虚部 b 都是浮点型。
- 复数对象有两个属性`real`和`imag`用于查看实部和虚部。

In [29]:



```
x=1+2j  
print(x, x.real, x.imag)
```

(1+2j) 1.0 2.0

In [30]:



```
x=complex(3, 4)  
print(x, x.real, x.imag)
```

(3+4j) 3.0 4.0

● 基本数值类型

● 数值类型实例

int	float	complex
10	0.0	3.14j
100	15.20	45.j
-786	-21.9	9.322e-36j
0o10	32.3e2	.876j
-0o10	-90.	-.6545+0j
-0x260	-32.54e100	3e+26j
0x69	70.2e12	4.53e-7j

● 基本数据类型-字符型数据

- 在Python中，字符串是用**单引号**(')、**双引号**(")、**三单引号**('')或**三双引号**('\"\"')括起来，且必须配对使用，字符串被定义为引号之间的字符集合，Python中的字符串属于**不可变序列**。

(1) 创建字符串

只要为变量分配一个用字符串界定符括起来的字符序列即可创建一个字符串。

```
In [38]: ▶ var1="Hello World!"  
print(var1)
```

```
Hello World!
```

➤ 多行字符串：

对于多行字符串，可以使用三个单引号'''或三个双引号"""来取代原有的引号的方式加以实现。

多行字符串会自动在每行末尾添加了'\n'作为换行符，但通过三个引号的方式可以让程序编写人员看起来更清晰。

➤ 多行字符串:

```
In [21]: ▶ print('''Python是一门编程语言  
Python能做数学运算  
要做好练习  
''')  
print('Python是一门编程语言\nPython能做数学运算\n要做好练习\n')
```

Python是一门编程语言
Python能做数学运算
要做好练习

Python是一门编程语言
Python能做数学运算
要做好练习

● 基本数据类型-列表 (list)

- 列表是Python里面的容器之一，由方括号和方括号括起来的数据构成。里面的数据可以是**整数、浮点数、字符串**，也可以是**另一个列表或者其它的数据结构**。列表里面的**每一项**叫作列表的一个**元素**，**每个元素**之间使用**英文逗号**隔开，例如：

`list_1 = [1, 2, 3, 4, 5]` #列表里面有5个元素，全部是数字

`list_2 = ['abc', 'x', '', 'kkk']` #列表里面有4个元素，全部是字符串

`list_3 = []` #空列表

`list_4 = [123, 'xyz', 3.14, [1, 2, 'yy']]` #由多种元素组合起来的列表

● 基本数据类型-元组 (tuple)

- **元组**是Python里面的容器之一，由**小括号**和小括号括起来的数据构成。它的外型和列表非常像，只不过列表使用的是方括号，元组使用的是小括号。“元组”中的“元”和“二元一次方程”中的“元”是同一个意思，“组”就是组合的意思。

`tuple_1 = (1, 2, 3, 4, 5)` #元组里面有5个元素，全部为数字

`tuple_2 = ('abc', 'x', '', 'kkk')` #元组里面有4个元素，全部是字符串

`tuple_3 = ()` #空元组

`tuple_4 = (123, 'xyz', [1, 't', 'z'], ('o', 'pp'))` #由多种元素组合起来的元组

元组和列表的区别

列表: $L=[1, 2, 5, 7]$

元组: $T=(1, 2, 5, 7)$

- 列表生成以后还可以往里面继续添加数据，也可以从里面删除数据；但是元组一旦生成就不能修改。如果它里面只有整数、浮点数、字符串、另一个元组，就既不能添加数据，也不能删除数据，还不能修改里面数据的值。但是如果元组里面包含了一个列表，那么这个元组里面的列表依旧可以变化。

$TT=([a, b, c], 12, \text{"Hello"})$

元组和列表的数据读取

- 与字符型数据类似，列表和元组数据结构中，取任何一个元素，都可以直接使用变量名[下标]。
- 左边下标都是从0开始的，第0个元素就是指最左边的元素，第1个元素就是指左边第二个元素，以此类推。
- 右边下标是从-1开始的，-1表示右边一个元素，-2表示右边第2个元素，-3表示右边第3个元素，以此类推。

```
example_list = ['我', '是', '列', '表']
```

在列表中，第0个元素是“我”字，第1个元素是“是”字，以此类推。

```
example_tuple = ('我', '是', '元', '组')
```

在元组中，第0个元素是“我”字，第1个元素是“是”字，以此类推。

元组和列表的数据读取



```
In [69]: ▶ example_list = ['我', '是', '列', '表']  
example_tuple = ('我', '是', '元', '组')  
print((example_list[1]))  
print(example_tuple[2])  
print(example_list[-2])  
print(example_tuple[-3])
```

是
元
列
是

● 字典

字典就是使用大括号括起来的键（Key）值（Value）对（Key-Value对）。每个键值对之间使用英文逗号分隔，每个Key与Value之间使用英文冒号分隔。

例如：

```
dict_1 = {'superman': '超人是一个可以在天上飞的两足兽', '天才': '天才跑在时代的前面，把时代拖得气喘吁吁。', 'xx': 0, 42: '42是一切的答案'}
```

● 基本数据类型-字典

- **Key**可以使用**中文、英文或者数字**，但是**不能重复**。**Value**可以是任意**字符串、数字、列表、元组或者另一个字典**，**Value可以重复**。
- 可以**通过Key**来从字典中**读取对应的Value**，有3种主要的格式：

变量名[key]

变量名.get(key)

变量名.get(key, '在找不到key的情况下使用这个值')

```
In [164]: dict1={1:"good",2:"bad",3:"not good"}
          value1=dict1.get(1,"1")
          print(value1)
```

good

- 集合是使用大括号括起来的各种数据，可以看作没有Value的字典。集合里面的元素不能重复。集合也是无序的。
- 集合最大的应用之一就是去重。例如，把一个带有重复元素的列表先转换为集合，再转换回列表，那么重复元素就只会保留一个。把列表转换为集合需要使用set()函数，把集合转换为列表使用list()函数：

```
In [71]:  duplicated_list = [3, 1, 3, 2, 4, 6, 6, 7, 's', 's', 'a']
          unique_list = list(set(duplicated_list))
          print(unique_list)

[1, 2, 3, 4, 's', 6, 7, 'a']
```

Python运算符

运算符是对操作数进行运算的符号。 Python语言的运算符按照它们的功能可分为：

- (1) 算术运算符 (`+`、`-`、`*`、`/`、`**`、`//`、`%`) 。
- (2) 关系运算符 (`>`、`<`、`>=`、`<=`、`==`、`!=`) 。
- (3) 逻辑运算符 (`and`、`or`、`not`) 。
- (4) 位运算符 (`<<`、`>>`、`~`、`|`、`^`、`&`) 。
- (5) 赋值运算符 (`=`、复合赋值运算符) 。
- (6) 成员运算符 (`in`、`not in`) 。
- (7) 同一运算符 (`is`、`is not`) 。
- (8) 下标运算符 (`[ ]`) 。
- (9) 其他 (如函数调用运算符 `()`) 。

几个相关概念：

- ✓ 目（操作数个数）：单目和双目
- ✓ 优先级
- ✓ 结合性
- ✓ 表达式：表达式是变量、数字、运算符、函数、括号等构成的有意义组合体，表达式的返回值是一个单一的数字。

在一个表达式中，Python会根据运算符的优先级从高到低进行运算：

生成式>函数调用、下标、切片> 算术运算符>位运算符>
关系运算符>逻辑运算符>lambda表达式

运算符的优先级

算术>关系>逻辑
括号优先

1. 算术运算符和算术表达式

运算符	含义	优先级
+	加法	这些运算符的优先级相同， 但比下面的运算符优先级低
-	减法	
*	乘法	
/	除法	
//	取整除	这些运算符的优先级相同， 但比上面的运算符优先级高
**	幂运算	
%	取模	

1. 算术运算符和算术表达式

表 1-4-1 算术运算符

运算符	描述	实例
+	两个对象相加	10+20 输出结果 30
-	两个对象相减	10-20 输出结果 -10
*	两个数相乘	10*20 输出结果 200
/	x 除以 y	20/10 输出结果 2
%	除法的余数	20%10 输出结果 0
**	x 的 y 次幂	2**3 为 2 的 3 次方， 输出结果 8
//	取整除，商的整数部分	9//2 输出结果 4

运算符与表达式

1. 算术运算符和算术表达式

- 数值的除法 (/) 总是返回一个浮点数，要获取整数使用//操作符。
- 混合计算时，Python会把整型转换成为浮点数。

假设变量a为10，变量b为21

算数符	描述	实例
+	加-两个对象相加	a+b结果31
-	减-得到一个负数或者一个数减去另一个数	a-b结果-11
*	乘-两个数相乘或返回一个被重复若干次的字符串	a*b结果210
/	除-X除以Y	b/a结果为2.1
%	取模-返回除法的余数	b%a结果为1
**	幂-返回X的Y次幂	a**b结果为10的21次方
//	取整数-返回商的整数部分	b//a结果为2

不同数据类型在一起进行算术运算时，按照下面原则进行隐式类型转换：

- 当有一个操作数的类型是**复数**时，其它数据都自动**转换成复数**类型。
- 当有一个操作数的类型是**小数**时，其它数据都自动**转换成小数**类型。
- 不支持数字和字符之间的隐式转换。
- 数值的除法（/）总是返回一个浮点数，要获取整数使用//操作符。
- **混合计算**时，Python会把**整型转换**成为**浮点数**。

运算符与表达式



- **运算符实现乘方运算，其优先级高于*和/。
- 注意运算符的多重含义。

```
In [85]: 4*3**2
```

```
Out[85]: 36
```

```
In [86]: "a"*10
```

```
Out[86]: 'aaaaaaaaaa'
```

2. 关系运算符和关系表达式

运算符	含义	优先级	结合性
>	大于	这些运算符的优先级相同， 但比下面的运算符优先级低	左结合
>=	大于等于		
<	小于		
<=	小于等于		
==	等于	这些运算符的优先级相同， 但比上面的运算符优先级高	
!=	不等于		
<>	不等于		

➤ 关系运算:

(又称**比较运算符**) 通过关系运算符将两个可做对比的操作数连接起来, 寻找其中的关系, 其结果为**布尔值**。

● 关系运算符

运算符	示 例
==	1 == 2为False, 'ABC' == 'ABC' 为True
!=	2 != 3 为True, 'ABC' != 'abc' 为True
>	10 > 2 为True
>=	'20' >='2C' 为False, 字符串按对应位字符的ASCII码大小比较
<	'20' < '19' 为False
<=	1 <= 2为True

2. 关系运算符和关系表达式

表 1-4-2 关系运算

数学符号	Python 关系运算符 号	说明	举例
$>$	$>$	大于	$5>2$
$\geq$	$\geq$	大于或等于	$4\geq 3$
$<$	$<$	小于	$5<6$
$\leq$	$\leq$	小于或等于	$5\leq 6$
$=$	$==$	等于	$5==5$
$\neq$	$!=$	不等于	$2!=3$

2. 关系运算符和关系表达式:

关系运算符——注意符号区别

注意正确使用 “=” 与 “==”

```
a,b=0,1
if a=b:
    print("a equal to b")
else:
    print("a not equal to b")
```

```
a,b=0,1
if a==b:
    print("a equal to b")
else:
    print("a not equal to b")
```

语法错误: **invalid syntax**

运行结果: **a not equal to b**

2. 关系运算符和关系表达式

注：Python采用None表示空值，表示没有值，属于一个特殊的值，而0为一个数值，因此二者并不相等。

```
In [22]: ▶ print(not 0)  
          print(not 3)
```

```
True  
False
```

3. 逻辑运算符和逻辑表达式

● 逻辑运算符

运算符	含义	优先级	结合性
not	逻辑非	 高 低	右结合
and	逻辑与		左结合
Or	逻辑或		

3. 逻辑运算符和逻辑表达式

逻辑运算符用来对布尔值进行与、或、非等逻辑运算，运算最终结果是布尔值True和False。

其中，“非”是单目运算符，布尔“与”和“或”为双目运算符。

逻辑运算符的操作数都应该是布尔值，如果是其它类型的值，应该能够转换为布尔值才能进行运算。

and: 逻辑与; or: 逻辑或; not: 逻辑非。

3. 逻辑运算符和逻辑表达式

● 逻辑表达式及短路运算：

(1) 对于与运算 `a and b`

如果a为真，继续计算b，b将决定最终整个表达式的真值，所以，结果为b的值。

如果a为假，无需计算b，就可以得知整个表达式的真值为假，所以，结果为a的值。

```
In [6]: ▶ True and 0
```

```
Out[6]: 0
```

```
In [7]: ▶ 1 False and 12
```

```
Out[7]: False
```

3. 逻辑运算符和逻辑表达式

● 逻辑表达式及短路运算：

(2) 对于或运算 `a or b`

如果a为真，无需计算b，就可得知整个表达式的真值为真，所以结果为a的值。

如果a为假，继续计算b，b将决定整个表达式最终的值，所以结果为b的值。

```
In [8]: ► True or 0
```

```
Out[8]: True
```

```
In [9]: ► False or 12
```

```
Out[9]: 12
```

4. 位运算符:

位运算符将数字转换成二进制数来进行运算，位运算符只能用于整型或字符串型数据，不能用于浮点型数据。

运算符	描述	实例
&	按位与运算	60 & 13 = 12
 	按位或运算	60 13 = 61
^	按位异或运算	60 ^ 13 = 49
~	按位取反运算	~60 = -61
<<	左移运算	60 << 2 = 240
>>	右移运算	60 >> 2 = 15

5. 身份运算符

- 身份比较符用于比较两个对象的内存位置是否相同, 使用`id()`函数来确定。

运算符	含义	优先级	结合性
<code>is</code>	相同	相同	左结合
<code>is not</code>	不相同		

- `is`检查用来运算的两个变量是否引用同一对象, 如果相同返回`True`, 不相同则返回`False`。
- `is not`检查用来运算的两个变量是否不是引用同一对象, 如果不是同一个对象返回`True`, 否则返回`False`。

5. 身份运算符

运算符	描述	实例
is	is 是判断两个标识符是不是引用自同一个对象	x is y ，类似 id(x) == id(y) ，如果引用的是同一个对象则返回 True ，否则返回 False
is not	is not 是判断两个标识符是不是引用自不同对象	x is not y ，类似 id(a) != id(b) 。如果引用的不是同一个对象则返回结果 True ，否则返回 False 。

5. 身份运算符

```
In [16]: ▶ x=y=2.5  
z=2.5  
x is not z
```

```
Out[16]: True
```

6. 成员运算符和成员表达式

- 成员测试符用于查找对象是否在列表、元组、字符串、集合和字典等序列数据中，其运算结果仍是逻辑值。

运算符	含义	优先级	结合性
In	存在	相同	左结合
not in	不存在		

成员运算符	逻辑表达式	描述
in	x in y	如果 x 在 y 序列中返回 True , 否则返回 False 。
not in	x not in y	如果x不在 y 序列中返回 True , 否则返回 False

6. 成员运算符和成员表达式

```
In [10]: ► 'a' in 'abcd'
```

```
Out[10]: True
```

```
In [11]: ► 'ac' in 'abcd'
```

```
Out[11]: False
```

6. 成员运算符和成员表达式

- `not in` 运算符用于在指定的序列中查找某个值是否存在，不存在返回True，存在返回False。

```
In [12]: ► 'a' not in 'b, c, d'
```

```
Out[12]: True
```

```
In [14]: ► 3 not in (1, 2, 3, 4)
```

```
Out[14]: False
```

运算符与表达式

- Python语言支持链式关系表达式，相当于多个表达式之间逻辑与的关系。

$1 < 2 < 3$ #等价于 $1 < 2$ and $2 < 3$

$2 \geq 1 > 10$ #等价于 $2 \geq 1$ and $1 > 10$

赋值语句

1. 赋值

赋值语句的语法格式如下：变量=表达式

2. 复合赋值：复合赋值运算中，解释器是先算复合赋值运算符右边的表达式，再进行复合赋值运算。如： $a-=b$ 等效于 $a=a-b$

3. 多变量赋值

链式赋值：变量1=变量2=……=变量n=表达式

同步赋值：变量1, 变量2, …… , 变量n=表达式1, 表达式2, …… , 表达式n

7. 赋值运算符和赋值表达式

● 赋值运算符

赋值运算符用 “=” 表示。

● 赋值表达式

变量 = 表达式

等号的左边必须是变量，右边是表达式。

7. 赋值运算符和赋值表达式

●链式赋值

在Python中，可通过链式赋值将同一个值赋给多个变量的，一般形式为：

```
In [88]: ▶ x=y=5  
          print(x, y, sep=', ')  
  
          5, 5
```

7. 赋值运算符和赋值表达式

●多变量并行赋值

变量1, 变量2, ..., 变量n=表达式1, 表达式2, ..., 表达式n

变量个数要与表达式的个数一致，其过程为：首先计算表达式右边n个表达式的值，然后同时将表达式的值赋给左边的n个变量。

```
In [89]: x, y, z=3, 6, 9  
         print(x, y, z, sep=',')
```

3, 6, 9

```
In [98]: x=20  
         x, x=3, x*3  
         print(x)
```

60

7. 赋值运算符和赋值表达式

● 多变量并行赋值

采取并行赋值，可以使用一条语句就可以交换两个变量的值： $x, y=y, x$ 。

```
In [102]: x, y=4, 10  
          print("交换前的值为: x=", x, "y=", y)  
          x, y=y, x  
          print("交换后的值为: x=", x, "y=", y)
```

交换前的值为: x= 4 y= 10

交换后的值为: x= 10 y= 4

7. 赋值运算符和赋值表达式

● 复合的赋值运算符

赋值运算符“=”与7种算术运算符（+、-、*、/、//、**、%）和5种位运算符（>>、<<、&、^、|）结合构成12种复合的赋值运算符。

例如：

$a+=3$

等价于

$a=a+3$

$a*=a+3$

等价于

$a=a*(a+3)$

$a\%=3$

等价于

$a=a\%3$

注意：“ $a*=a+3$ ”与“ $a=a*a+3$ ”是不等价的，“ $a*=a+3$ ”等价于“ $a=a*(a+3)$ ”，这里括号是必需的。

➤ 复合赋值运算符

运算符	含义	举例	等效于
+=	加法赋值字符串 拼接	sum += itemaStr += "Foo"	sum = sum + itemaStr = aStr + "Foo"
-=	减法赋值	count -= 1	count = count - 1
*=	乘法赋值	x *= y + 5	x = x *(y + 5)
/=	除法赋值	x /= y - z	x = x/(y - z)
//=	整除赋值	x //= y - z	x = x//(y - z)
%=	取模赋值	x %= 2	x = x%2
**=	幂运算赋值	x **= 2	x = x ** 2
<<=	左移赋值	x <<= y	x = x << y
>>=	右移赋值	x >>= y	x = x >> y
&=	按位与赋值	x &= y	x = x & y
 =	按位或赋值	x = y	x = x y
^=	按位异或赋值	x ^= y	x = x^y

7. 赋值运算符和赋值表达式

● 复合的赋值运算符:

```
In [104]: ▶ a, b=3, 5  
          a+=b  
          print(a)  
  
8
```

```
In [105]: ▶ a=7  
          a*=a+3  
          print(a)  
  
70
```

➤ 运算符的优先级

优先级	运算符	描述
1	**	指数 (最高优先级)
2	~ + -	按位翻转，正负号
3	* / % //	乘、除、取模和取整除
4	+ -	加法减法
5	>> <<	右移、左移运算符
6	&	按位与
7	^	位运算符
8	<= < > >=	比较运算符
9	== !=	等于运算符
10	= %= /= //= -= += *= **=	赋值运算符
11	is is not	身份运算符
12	in not in	成员运算符，判断一个字符串是否包含于另一个字符串
13	not or and	逻辑运算符

7. 自动类型转换:

在Python中，同一个表达式允许不同类型的数据参加运算，这就要求在运算之前，先将这些不同类型的数据转换成同一类型，然后再进行运算。

```
In [17]: ▶ type(10/4*4)
```

```
Out[17]: float
```

8. 强制类型转换:

当自动类型转换达不到转换需求时, 可以使用类型转换函数, 将数据从一种类型强制 (或称为显式) 转换成另一种类型。

函数	功能描述
<u>int</u> (x)	将x转换为整数
<u>float</u> (x)	将x转换为浮点数
<u>complex</u> (x)	将x转换为复数, 其中实部为x, 虚部为0
<u>complex</u> (x, y)	将x、y转换为复数, 其中实部为x, 虚部为y
<u>str</u> (x)	将x转换为字符串
<u>chr</u> (x)	将一个整数转换为一个字符, 整数为字符的ASCII编码
<u>ord</u> (x)	将一个字符转换为它的ASCII编码的整数值
<u>hex</u> (x)	将一个整数转换为一个十六进制字符串
<u>oct</u> (x)	将一个整数转换为一个八进制字符串
<u>eval</u> (x)	将字符串str当做有效表达式求值, 并返回计算结果

Python中的函数分为内置函数和模块函数。

1. 内置函数

- 内置函数 (BIF, built-in functions) 是Python内置对象类型之一, 不需要导入任何模块即可使用一类函数。
- 执行`dir(__builtins__)`可以列出所有的内置函数;
- 使用`help(函数名)`可以查看某个函数的用法;

`abs(x)`: 返回数字`x`的绝对值或复数`x`的模;

`bin(x)`、`oct(x)`、`hex(x)`: 把整数`x`转换为二进制、八进制、十六进制的表示形式。

`bytes(x)`: 把指定对象`x`转换为字节串表示形式。

- `float(x)`: 把整数或字符串`x`转换为浮点数;
- `int(x[, d])`: 返回实数 (`float`)、分数 (`Fraction`) 或高精度实数 (`Decimal`) `x`的整数部分, 或把`d`进制的字符串`x`转换为十进制并返回, `d`默认为十进制;
- `chr(x)`: 返回Unicode编码为`x`的字符;
- `ord(x)`: 返回1个字符`x`的Unicode编码;
- `isinstance(obj, type)`: 测试对象`obj`是否属于指定类型`type`;
- `type(obj)`: 返回对象`obj`的类型;
- `max(x)`、`min(x)`: 返回可迭代对象`x`中的最大值、最小值;
- `len(obj)`: 返回对象`obj`包含的元素个数;

- `reversed(seq)`: 返回`seq`中所有元素逆序后的迭代器对象;
- `sorted(iterable, key=None, reverse=False)`: 返回排序后的列表, 其中`iterable`表示要排序的序列或迭代对象, `key`用来指定排序规则或依据, `reverse`用来指定升序或降序, 默认为升序。该函数不改变`iterable`内任何元素的顺序;
- `sum(x, start=0)`: 返回序列`x`中所有元素之和;
- `exit()`: 退出当前解释器环境;

Python中的函数分为**内置函数**和**模块函数**。

2. 模块函数

1) **math**模块

math模块提供了众多功能强大的数学函数，可以有效提高编程效率，使用这些函数时，需要使用import math导入math模块。

常数	描 述
math.pi	圆周率pi
math.e	自然常数e
math.tau	数学常数 τ
math.inf	正无穷大
math.nan	非浮点数标记，NaN(Not a Number)

➤ math数学库函数及常量

函数或常量	含义
<code>math.pi</code>	圆周率 π
<code>math.e</code>	自然常数 e
<code>math.ceil(x)</code>	对浮点数向上取整
<code>math.floor(x)</code>	对浮点数向下取整
<code>math.fabs(x)</code>	求 x 的绝对值
<code>math.pow(x,y)</code>	计算 x 的 y 次方
<code>math.log(x)</code>	以 e 为基数的对数
<code>math.log10(x)</code>	以10为基数的对数
<code>math.sqrt(x)</code>	平方根
<code>math.exp(x)</code>	e 的 x 次幂
<code>math.degrees(x)</code>	将弧度值转换为角度值
<code>math.radians(x)</code>	将角度值转换为弧度值
<code>math.sin(x)</code>	正弦函数
<code>math.cos(x)</code>	余弦函数
<code>math.tan(x)</code>	正切函数
<code>math.asin(x)</code>	反正弦函数, $x \in [-1.0, 1.0]$
<code>math.acos(x)</code>	反余弦函数, $x \in [-1.0, 1.0]$
<code>math.atan(x)</code>	反正切函数

➤ math数学库函数及常量

```
In [23]: ▶ import math  
print(math.pi)  
print(math.e)  
print(math.tau)  
print(math.inf)  
print(math.log10(100))
```

```
3.141592653589793  
2.718281828459045  
6.283185307179586  
inf  
2.0
```

2) random模块

在编写程序时，经常需要提供一些随机数据。大多数编程语言提供了生成伪随机数的函数，在Python中这类函数被封装在random模块中。

函数名	说明	举例
random.random()	返回[0.0,1.0]区间内的一个随机小数	random.random() 返回 0.9523521796999529
random.uniform(a,b)	返回[a,b]区间内的一个随机小数	random.uniform(1,3) 返回 2.3403540401554146
random.randint(a,b)	返回[a,b]区间内的一个随机整数	random.randint(1,3) 返回2
random.randrange([start],end,[step])	返回[start,end)区间内的一个整数，start和step默认都是1	random.randrange(1,10) 返回 5
random.choice()	随机返回给定序列中的一个元素	random.choice(['a','b','c']) 返回 'c'
random.shuffle(x,[random])	将可变序列的所有元素随机排列	random.shuffle([1,2,3,4]) 返回 [1, 4, 3, 2]

